

Week 03: Bellman Error and Deep

Q-Networks

Replacing Tabular Lookups with Parametric Function Approximators

Reinforcement Learning & Robot Learning Core Curriculum

Presented by **DGIST ISL Team**

DGIST Intelligent Systems and Learning Laboratory

2026.08.21

Core Concepts: Bootstrapped Bellman Loss · Target Networks · Experience Replay · Diagnostic Baselines

The Scaling Wall: Tabular to Function Approximation

Continuous and high-dimensional states require generalizable parameterization

Tabular Q-Learning (Week 02)

Lookup Table: $Q[s, a] \in \mathbb{R}^{|S| \times |A|}$

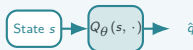
- Updates affect **strictly one cell** (s, a) at a time.
- Zero generalization across similar states ($s = 1.01$ vs $s = 1.02$).
- **Curse of Dimensionality:** Continuous robot states make tables impossible.



Parametric Q-Approximation (Week 03)

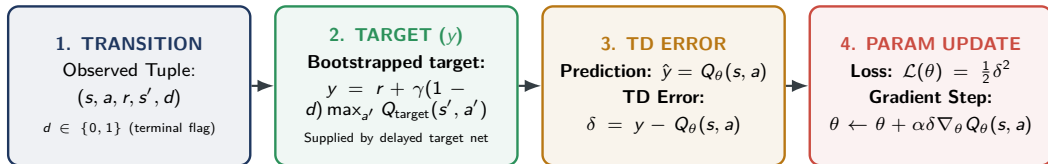
Model Prediction: $Q_{\theta}(s, a) \approx Q^*(s, a)$ with weights θ

- An update to θ **generalizes across state regions**.
- Scales to continuous observations, embeddings, and raw images.
- **Tradeoff:** Changing θ shifts values for *many* states simultaneously.



Method Architecture: Constructing the Bellman Loss

Temporal-Difference (TD) learning transforms dynamic programming into supervised regression



Non-Terminal Case ($d = 0$)

Target $y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$. Future values are discounted and bootstrapped from next state.

Terminal Case ($d = 1$)

Target $y = r$. At goal/failure termination, future return is strictly zero. Never bootstrap across true episode boundaries!

Concrete Walkthrough: LineWorld Arithmetic

Hand-traceable example: how the Bellman error translates into exact numerical loss

Non-Terminal Step: State 2 $\xrightarrow{\text{right}}$ 3

Current State: $s = 2, a = \text{right}, r = -0.01, s' = 3, \gamma = 0.9$

Online Prediction: $Q_{\theta}(2, \text{right}) = 0.10$

Target Network values at $s' = 3$:

- $Q_{\text{target}}(3, \text{left}) = 0.20$
- $Q_{\text{target}}(3, \text{right}) = \mathbf{0.50} \implies \max = 0.50$

Target: $y = -0.01 + 0.9(0.50) = \mathbf{0.44}$

TD Error: $\delta = 0.44 - 0.10 = \mathbf{+0.34}$

Squared Error: $\delta^2 = \mathbf{0.1156}$

Optimization Loss: $\mathcal{L} = \frac{1}{2}\delta^2 = \mathbf{0.0578}$

Terminal Step: State 3 $\xrightarrow{\text{right}}$ Goal(4)

Current State: $s = 3, a = \text{right}, r = +1.00, s' = 4(\text{Goal}), d = \text{True}$

Online Prediction: $Q_{\theta}(3, \text{right}) = 0.50$

Future value discarded: $d = 1 \implies$ no bootstrap

Target: $y = r = \mathbf{+1.00}$

TD Error: $\delta = 1.00 - 0.50 = \mathbf{+0.50}$

Squared Error: $\delta^2 = \mathbf{0.2500}$

Optimization Loss: $\mathcal{L} = \frac{1}{2}\delta^2 = \mathbf{0.1250}$

Outcome: Network parameters increase $Q_{\theta}(3, \text{right})$ toward 1.0.

The Deadly Triad & DQN's Operational Components

Function approximation + bootstrapping + off-policy learning can produce unstable or divergent updates

1. Replay Buffer

Problem: Training only on the newest adjacent transitions can make updates dominated by recent trajectory segments.

Mechanism: Store transitions in FIFO buffer $\mathcal{D}$; sample uniform random mini-batches.

Effect: Reduces local temporal adjacency and reuses data; sampled replay is not literally i.i.d.

2. Target Network

Problem: Chasing a moving target. Updating Q_θ simultaneously shifts $y = r + \gamma \max_a Q_\theta(s', a')$.

Mechanism: Separate target network Q_{θ^-} with frozen weights; sync $\theta^- \leftarrow \theta$ every C steps.

Effect: Slows one source of target motion; it does not eliminate feedback or guarantee convergence.

3. ϵ -Greedy Policy

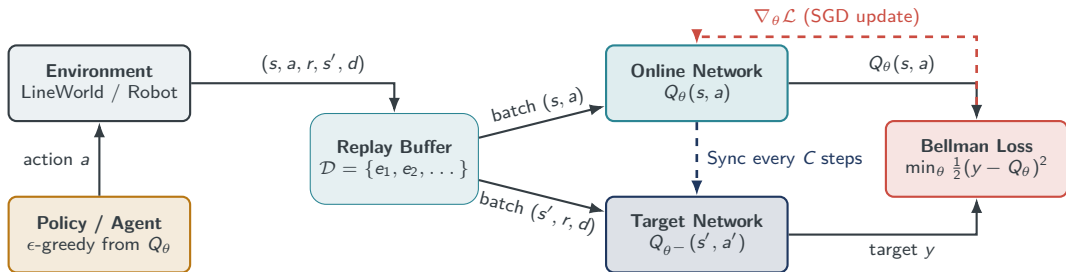
Problem: A greedy behavior policy may never sample actions that early estimates or tie-breaking disfavor.

Mechanism: Act randomly with prob ϵ ; greedily $\arg \max_a Q_\theta(s, a)$ with prob $1 - \epsilon$.

Effect: Gives each action nonzero local sampling probability; state-action coverage is not guaranteed.

The Complete Deep Q-Network (DQN) System Architecture

How environment interaction, replay buffer, online network, and target network interact



Code Lens: From Gradient Update to Evaluation

Transparent linear function approximation in examples/week-03

Linear Parametric Model

Parametric Form

$$Q_{\theta}(s, a) = w_{0,a} + w_{1,a} \cdot \left(\frac{s}{s_{\max}-1} \right)$$

- Feature vector for state s : $\phi(s) = [1.0, s/4.0]$.
- **Sparse Parameter Update:** Only weights for the executed action a receive non-zero gradients!
- $\nabla_{\theta_a} Q_{\theta}(s, a) = \phi(s)$.
- Weight update: $\theta_a \leftarrow \theta_a + \alpha \cdot \delta \cdot \phi(s)$.

Empirical Evaluation

Policy	Avg Return	Success Rate
Random Policy	+0.52	65%
Always Right (Heuristic)	+0.97	100%
Linear Q-Learner	+0.97	100%

Diagnostic Discipline

Never rely on Bellman loss alone. Low loss can coincide with poor policies if targets or sampled distributions are degenerate. Always evaluate episode returns.

Failure Modes in Value Function Approximation

Common engineering and theoretical pitfalls in DQN implementations

1. TARGET NETWORK LEAKAGE

Calculating targets directly with online weights Q_θ causes moving target oscillations and divergence. Target must use strictly detached $Q_{\theta-}$.

2. TERMINATION VS TRUNCATION

Dropping future return on timeout truncation ($t \geq T_{\max}$) corrupts value estimates. Only true MDP terminal states ($d = \text{True}$) zero out future value.

3. LOSS-POLICY MISALIGNMENT

A decreasing Bellman loss does *not* guarantee policy improvement. If the replay buffer has narrow coverage, the model fits irrelevant state subspaces.

4. OVERGENERALIZATION

Neural network weights update globally. An update at state s can degrade value accuracy at distant state s' , requiring replay buffers to retain past memories.

Robotics Bridge: From DQN to Physical Systems

Why continuous robot control adapts Bellman regression into Actor-Critic methods

Robotic Observation Spaces

- Joint positions, velocities, torques ($\mathbb{R}^7$).
- End-effector 6D poses & tactile feedback.
- Camera RGB-D streams ($224 \times 224 \times 4$).
- Language task embeddings (e.g., CLIP / T5).

High-Dimensional State

DQN's core insight (neural value approximation) is mandatory for robot state representation.

The Action Space Bottleneck

- DQN requires computing $\arg \max_{a \in A} Q(s, a)$.
- For continuous torque/velocity actions ($a \in \mathbb{R}^d$), discrete max optimization is intractable.

Transition to Actor-Critic (Weeks 5–7):

- ★ **Critic:** Evaluates Bellman error $Q_\phi(s, a)$.
- ★ **Actor:** Parametric policy $\pi_\theta(s)$ directly outputs continuous actions.

Week 03 Quiz: Core Concepts & Mechanisms

Self-check: verify fundamental understanding of value approximation

Q1. Role of the Target Network ($Q_{\theta-}$)

Why does DQN periodically freeze and copy weights into $Q_{\theta-}$ instead of using Q_{θ} for targets?

- A. To prevent overfitting to the training set.
- B. To stop the target from moving on every single gradient update, eliminating destructive positive feedback loops.
- C. To ensure actions are always chosen greedily.

Answer: B $\implies$ Stabilizes bootstrapping regression.

Q2. Bellman Loss vs. Policy Return

Can Bellman loss decrease while the agent's actual return in the environment remains near zero?

- A. No, loss is inversely proportional to return.
- B. Yes, if replay data is narrow/biased or targets are bad, the model can accurately fit useless or dead-end states.
- C. Yes, but only when $\gamma = 1.0$.

Answer: B $\implies$ Diagnostic metric discipline.

Week 03 Quiz: Target & TD Error Arithmetic

Verify hand-computations for terminal vs non-terminal Bellman targets

Problem 1: Non-Terminal Transition

Given: $\gamma = 0.9$, $r = -0.01$, $d = \text{False}$

$Q_{\theta}(2, \text{right}) = 0.10$

$Q_{\text{target}}(3, \text{left}) = 0.20$, $Q_{\text{target}}(3, \text{right}) = 0.50$

Calculate: Target y , TD Error δ , and Loss $\mathcal{L} = \frac{1}{2}\delta^2$.

Solution:

- $y = -0.01 + 0.9 \cdot \max(0.20, 0.50) = \mathbf{0.44}$
- $\delta = y - Q_{\theta}(2, \text{right}) = 0.44 - 0.10 = \mathbf{+0.34}$
- $\mathcal{L} = \frac{1}{2}(0.34)^2 = \mathbf{0.0578}$

Problem 2: Terminal Transition

Given: $r = +1.00$, $d = \text{True}$ (Goal reached)

$Q_{\theta}(3, \text{right}) = 0.50$

$Q_{\text{target}}(4, a') = \text{arbitrary values}$

Calculate: Target y , TD Error δ , and Loss $\mathcal{L} = \frac{1}{2}\delta^2$.

Solution:

- $d = \text{True} \implies$ Future value term is 0
- $y = r = \mathbf{+1.00}$
- $\delta = 1.00 - 0.50 = \mathbf{+0.50}$
- $\mathcal{L} = \frac{1}{2}(0.50)^2 = \mathbf{0.1250}$

Week 03 Quiz: Stabilizers & Implementation Traps

Key practical questions every RL practitioner must master

Q3. Experience Replay Mechanism

Why sample a replay buffer instead of only the newest step?

Replay reduces domination by adjacent recent transitions and reuses older data. Its samples are still not literally i.i.d.

Q4. Timeout Truncation Handling

What happens if you set $d = 1$ on step timeouts?

The agent mistakenly learns that states near timeout have zero future return, artificially penalizing long-horizon but viable paths.

Q5. Linear Approximator Scope

Does matching the heuristic in LineWorld prove deep RL?

No. It demonstrates a Bellman-error update in one 1D toy world; full DQN adds a neural network, replay, and a delayed target network.

Q6. Action Parameter Updates

Why are only parameters for the chosen action updated?

Because $Q_\theta(s, a)$ outputs distinct branches/weights per action; only θ_a produced the prediction being supervised.